7 Logic I

Wordcount: 1798

For the last few sections of work, we have been covering proofs extensively. Some of these proofs such as diagonalisation, are conceptually difficult but easy do. Others like some of the more intensive reductions are both conceptually difficult *and* difficult to do.

Personally, as a computer scientist and as a programmer, my instinct in these situations is to ask why do I have to do this. We have these incredible inventions in computers and so far the only thing they apparently can't do is determine properties about themselves. Surely, we can write some algorithm, let it run in the corner of a building somewhere and determine if something is true or not.

In fact, we can think of four possible questions that would be valuable to know about any question:

1. Is something true?
2. Can it be true?
3. Is it always true?
4. If something else is true, then is this true?

> **Note**
>
> Question 3 is the question that inspired Turing to invent his famous machines. It was known as the Entschiedungsproblem.
>
> The last thing we do in this course is answer this question.

7.1 Computational Logic

As you have hopefully realised, the study of mathematics is deeply related to logic. Even something as simple as finding the roots of an equation is tied to logic.

$$\begin{aligned}&(x-5)\cdot(x+5)=0\\ \implies\ &(x-5)=0 \text{ or } (x+5)=0\\ \implies\ &x=5 \text{ or } x=-5\end{aligned}$$

We use a rule in algebra called the zero product rule to perform the jump from step 1 to step 2.

$$\forall a,b\in\mathbb{R}, ab=0\leftrightarrow a=0\vee b=0$$

Formal Logic is the study of when and how things are true. Starting from some base set of 'axioms', we build up the theorems, lemmas, and propositions that guarantee certain truths about what we are doing. In real analysis, we started with the axioms of addition, multiplication and order. For *all* of modern mathematics, we use the axioms of Zermelo-Fraenkel Set Theory.

Even Turing Machines are built using ZFC set theory. The definition of states, alphabets and instructions are dependent on sets.

So to run an algorithm for our questions, a good place to start is with logic. There are many branches of logic, that determine what they can and can't represent. To begin with, lets begin with the simplest form of logic:

Propositional.

A Proposition is either True or False.

7.2 The formal language of propositional logic

To run an algorithm, we first need to be able to actually put it onto the tape of a Turing Machine. So before we begin with our algorithm, lets make sure we can actually write 'logic' onto our machine's tape.

7.2.1 Syntax

To start, we need to define an alphabet of symbols.

We have our true or false objects called propositions, $PROP$, and then the different ways we can connect them together, $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$.

Not every word created from this alphabet can have meaning. For example the word $\vee \vee \vee$ doesn't say anything.

To address this, we say a word w is a well-formed **formula** if

1. $w = P$ for any $P \in PROP$
2. $w = \neg\varphi$ where φ is another formula
3. $w = \varphi_1 \vee \varphi_2$ where φ_1 and φ_2 are other formulae.
4. $w = \varphi_1 \wedge \varphi_2$
5. $w = \varphi_1 \rightarrow \varphi_2$
6. $w = \varphi_1 \leftrightarrow \varphi_2$

Parsing Formula

We can even write a decision problem to detect if a word is a formula.

$$\text{PROP-WELL-FORMED} := \{w \mid w \text{ is a formula}\}$$

Our above definition is also called a grammar, but you may see it written more like this

```
<proposition> ::= [A-Z] // an uppercase roman letter
<logic_not> ::= '¬' <formula>
<logic_or> ::= <formula> '∨' <formula>
<logic_and> ::= <formula> '∧' <formula>
<logic_imply> ::= <formula> '→' <formula>
<logic_equiv> ::= <formula> '↔' <formula>
<formula> ::= <proposition> | <logic_or> | <logic_and> | <logic_imply> | <logic_equiv>
```

This process of writing a grammar is the same way we define programming languages.

1. C
2. Python

They just tend to be a bit longer. The study of parsing these grammars and detecting if something matches the grammar is another very rich branch of theoretical and practical computer science.

7.2.2 Semantics

Now that we can write a formula onto the tape of a Turing Machine, we need to actually provide some meaning to the symbols.

In Propositional Logic, we typically rely on truth tables to define the 'meaning' of symbols.

A	B	$\neg A$	$A \vee B$	$A \wedge B$	$A \to B$	$A \leftrightarrow B$
T	T	F	T	T	T	T
T	F	F	T	F	F	F
F	T	T	T	F	T	F
F	F	T	F	F	T	T

However, this a bit cumbersome and exhaustive to write out for larger and more complex formula.

Instead we are going to use a new object called a **propositional assignment** to simplify the notation.

$$v : PROP \mapsto \{T, F\}$$

In a formula φ, we will give each proposition P a value of either T for true, or F for false. Essentially a given assignment v is one particular row of a truth table.

We use the following notation to indicate the **satisfaction** (or **truth**) of a formula under an assignment

1. $v \vDash P \iff v(P) = T$
   - Conversly $v \nvDash P \iff v(P) = F$
2. $v \vDash \neg\varphi \iff v \nvDash \varphi$
3. $v \vDash \varphi_1 \vee \varphi_2 \iff v \vDash \varphi_1 \text{ or } v \vDash \varphi_2$
4. $v \vDash \varphi_1 \wedge \varphi_2 \iff v \vDash \varphi_1 \text{ and } v \vDash \varphi_2$
5. $v \vDash \varphi_1 \to \varphi_2 \iff v \nvDash \varphi_1 \text{ or } v \vDash \varphi_1 \wedge \varphi_2$
6. $v \vDash \varphi_1 \leftrightarrow \varphi_2 \iff v \vDash \varphi_1 \to \varphi_2 \text{ and } v \vDash \varphi_2 \to \varphi_1$

Excercise

Try not to think too hard about using the logical meaning of *or* to define the logical symbol $\vee$. We distinguish between the two by giving the symbol $\vee$ the name disjunction rather then or.

An interesting property of logical symbols however is that we can often define the meaning of later symbols using earlier ones. We can see parts of this in our definitions of satisfaction for implication ($\to$) and equivalence ($\leftrightarrow$).

What is the smallest number of logical symbols do we need to be able to define every other symbol?

This question is very important to the construction of digital computers as all we would have to do is physically build this minimal set, and then every other operation (gate) could be defined using them.

Using these semantics, we can actually answer our first question from earlier:

1. Is something true?

In our new language:

1. Is a formula φ true under assignment v?

As a language

$$\text{PROP-TRUTH} := \{\langle v, \varphi\rangle \mid v \vDash \varphi\}$$

The Turing Machine to decide this language is then

define M_{PT} on $\langle v, \varphi\rangle$:
  if $\varphi = P$ for any $P \in PROP$:
    if $v(P) = T$ then Accept else Reject
  if $\varphi = \neg\phi$:
    return not $M_{PT}(\langle v, \phi\rangle)$
  if $\varphi = \phi_1 \vee \phi_2$:
    return $M_{PT}(\langle v, \phi_1\rangle) = 1$ or $M_{PT}(\langle v, \phi_2\rangle) = 1$
  if $\varphi = \phi_1 \wedge \phi_2$:
    return $M_{PT}(\langle v, \phi_1\rangle) = 1$ and $M_{PT}(\langle v, \phi_2\rangle) = 1$
  if $\varphi = \phi_1 \to \phi_2$:
    return $M_{PT}(\langle v, \neg\phi_1\rangle) = 1$ or ($M_{PT}(\langle v, \phi_1\rangle) = 1$ and $M_{PT}(\langle v, \phi_2\rangle) = 1$)
  if $\varphi = \phi_1 \leftrightarrow \phi_2$:
    return $M_{PT}(\langle v, \phi_1 \to \phi_2) = 1$ and $M_{PT}(\langle v, \phi_2 \to \phi_1\rangle) = 1$

The machine M_{PT} makes a fairly big assumption. We know that we can represent a formula φ as a string $\langle\varphi\rangle$ using our earlier syntax, but can we actually represent an assignment?

Looking at our original definition of v we have

$$v : PROP \mapsto \{T, F\}$$

We already know that $PROP$ must be a finite set of symbols, else it wouldn't be part of our syntax for prop logic. If we state that our propositions are always in *lexographical* order (A, B C, …, Y, Z, AA, BB, …) then the assignment function is just a list of True / False values of the same size as $PROP$.

Or more eloquently

$$\langle v(P)\rangle := \begin{cases} 1 & \text{if } v(P) = T \\ 0 & \text{if } v(P) = F \end{cases}$$

$$\langle v\rangle := \langle v(A)\rangle\#\langle v(B)\rangle\#\ldots\#\langle v(AA)\rangle\#\ldots$$

Tip

If you wanted to more space efficient, each proposition is only ever a single bit so it is a fixed width representation. That means we can drop the seperator $\#$ from the string and just have

$$\langle v\rangle := \langle v(A)\rangle\langle v(B)\rangle\ldots\langle v(AA)\rangle\ldots$$

or a binary string of length $|PROP|$.

7.3 Properties of Propositional Logic

With an established definition of truth, we can now start to formulate our other questions in a logic friendly fashion.

As a refresher, here are questions we want answers to:

1. Is something true?
2. Can it be true?
3. Is it always true?
4. If something else is true, then is this true?

We already sorted out 1 in the previous section, so now we need to move onto 2.

7.3.1 Satisfiability

2. Can it be true

In our prop logic way of thinking

2. Is there an assignment v where the formula φ is true

This property is called **satisfiability**

$$\varphi \text{ is satisfiable} \iff \text{there is some assignment } v \text{ such that } v \vDash \varphi$$

> **Note**
>
> You may tempted to use the symbol $\exists$ and rewrite this as
>
> $$\varphi \text{ is satisfiable} \iff \exists v, v \vDash \varphi$$
>
> The $\exists$ symbol has a specific definition in First Order Logic - for the purposes of this course we want to avoid the overlap between a piece of *meta* notation and a piece of *object* notation.
>
> When we reach the definition of $\exists$ we'll discuss the differences in more depth.

Can we solve decide this property algorithmcally? Our language is

$$\text{PROP-SAT} := \{\langle \varphi \rangle \mid \varphi \text{ is satisfiable}\}$$

and our machine is

define M_{PS} on $\langle \varphi \rangle$:
  for every v:
    if $M_{PT}(\langle v, \varphi \rangle) = 1$ then
      Accept
  Reject

We use M_{PT} as a subroutine for this language, so to guarantee that M_{PS} is a decider, we need to make sure that our loop over the assignments is finite.

For every $P \in PROP$, we have two choices - True or False. So the number of total combinations is $2 \times 2 \times 2 \times \ldots \times 2$ for all the propositions, or

$$2^n$$

possible assignments.

The loop is finite in length, even if it is exponential.

Tip

PROP-SAT is more commonly just called SAT and is a very important problem in Computer Science.

You can take almost any problem and turn into into some version of a propositional formula. Then to find some answer for that problem, we just need to find an assignment for the formula, and then convert it back into it's original context.

Or to use the definitions we've established before, almost all *decidable* decision problems can be reduced to SAT even the ones about Turing Machines. So, if we had a polynomial time algorithm for SAT, then we would have a polynomial time algorithm for all of these other problems.

But we don't know if a polytime algorithm exists for SAT.
That sounds awfully familiar to another open problem in computer science…

7.3.2 Validity

With both question 1 and 2 out of the way, we come to:
"Is something always true"

Or in propositional logic:
"For every assignment, is the formula true"

This is called **validity** and has the following notation

$$\models \varphi \iff \text{For every assignment } v, v \models \varphi$$

Note

For the same reasons as $\exists$ and satisfiability, we will avoid using the $\forall$ symbol here until a technical definition is given.

The language of validity is

$$\text{PROP-VAL} := \{\langle \varphi \rangle \mid \models \varphi\}$$

We can construct the decider for PROP-VAL in an almost identical manner to PROP-SAT.

define M_{PS} on $\langle \varphi \rangle$:
  for every v:
    if $M_{PT}(\langle v, \varphi \rangle) = 0$ then
      Reject
  Accept

Validity is useful because it allows us to construct the various proof techniques we may use throughout proving more complex theorems. As an example, we'll prove that the propositional statement for 'proof-by-contrapositive' is valid.

Theorem: $(P \to Q) \leftrightarrow (\neg Q \to \neg P)$

RTP: $(P \to Q) \to (\neg Q \to \neg P)$

We can visualise our decision procedure using a truth table starting from P and Q moving right for each row.

P	Q	$\neg P$	$\neg Q$	$P \to Q$	$\neg Q \to \neg P$	$(P \to Q) \to (\neg Q \to \neg P)$
T	T	F	F	T	T	T
T	F	F	T	F	F	T
F	T	T	F	T	T	T
F	F	T	T	T	T	T

7.3.3 Concequence

A good instinct at this point might to be argue that nothing we have done so far is actually a ‘proof’ in the traditional sense. Going line by line, making an observation, and then evaluating towards a new statement. Instead our algorithm here is just a brute force method to test that a formula is always true.

This is correct.

What we consider to be a proof is the very last question from the beginning.
“If this is true, then that is true”

In propositional logic:
“If the formula ψ is true, then the formula φ is true”

This property is called **logical concequence**, or just concequence.

$$\psi \vDash \varphi \iff \text{for any } v, \text{ if } v \vDash \psi \text{ then } v \vDash \varphi$$

This gives concequences like

$$A \wedge B \vDash A$$

but what if we would like something more complex?

We can extend our definition of concequence to use a finite set Λ of true formula instead.

$$\Lambda = \{\psi_1, \psi_2, \ldots, \psi_n\} \vDash \varphi \iff \text{for any } v, \text{ if every } v \vDash \psi_i \text{ then } v \vDash \varphi$$

We can simplify this into just

$$\Lambda \vDash \varphi \iff \text{for any } v, \text{ if } v \vDash \bigwedge \Lambda \text{ then } v \vDash \varphi$$

The language is

$$\text{PROP-CONQ} := \{\langle \Lambda, \varphi \rangle \mid \Lambda \vDash \varphi\}$$

The decider is bit more complex then for validity and satisifiability, but not by much.

define M_{PC} on $\langle \Lambda, \varphi \rangle$:
  for every v:
    if $M_{PT}(\langle v, \bigwedge \Lambda \rangle) = 1$ then
      if $M_{PT}(\langle v, \varphi \rangle) = 0$ then

Reject

Accept

We are going to reuse the proof-by-contrapostive idea to visualise concequence in what is probably a more natural proof, but only in the one direction.

Theorem: $P \to Q \vDash \neg Q \to \neg P$

Let v be an assignment such that $v \vDash P \to Q$

$$
\begin{aligned}
& v \vDash P \to Q \\
\iff & v \vDash \neg P \text{ or } v \vDash P \land Q \\
\iff & v \vDash \neg P \lor (P \land Q) \\
\iff & v \vDash (\neg P \lor P) \land (\neg P \lor Q) \\
\iff & v \vDash \neg P \lor Q
\end{aligned}
$$

$$
\begin{aligned}
& \text{Case I: Let } v \nvDash Q \\
\iff & v \vDash \neg P \lor Q \text{ and } v \vDash \neg Q \\
\iff & v \vDash (\neg P \lor Q) \land \neg Q \\
\iff & v \vDash (\neg P \land \neg Q) \lor (Q \land \neg Q) \\
\iff & v \vDash \neg P \land \neg Q \\
\iff & v \vDash \neg Q \to \neg P
\end{aligned}
$$

$$
\begin{aligned}
& \text{Case II: Let } v \vDash Q \\
\iff & v \nvDash \neg Q \\
\iff & v \vDash \neg Q \to \neg P \text{ vacously}
\end{aligned}
$$

Therefore by cases I and II, $P \to Q \vDash \neg Q \to \neg P$

Excercise

Using the above proof as a guide, try to follow the same process but for the other direction of proof by contraposition.

7.3.4 Concequence and Validity

Suspicously enough concequence seems to relate to validity in some manner. We can show that proof by contraposition works by *validity* and *concequence*. This seems to imply that there is some relationship that exists between these two properties.

This in fact true, and a very useful property in our algorithms for concequence.

$$\Lambda \vDash \varphi \iff \vDash \bigwedge \Lambda \to \varphi$$

Using this, we can use M_{PV} to solve PROP-CONQ by transforming $\langle \Lambda, \varphi \rangle$ into $\langle \bigwedge \Lambda \to \varphi \rangle$. In other words, we can prove that

$$\text{PROP-CONQ} \leq_m \text{PROP-VAL}$$

First however, we need to show that

$$\Lambda \vDash \varphi \iff \vDash \bigwedge \Lambda \to \varphi$$

Theorem: $\Lambda \vDash \varphi \iff \vDash \bigwedge \Lambda \to \varphi$

RTP: $\Lambda \vDash \varphi \implies \vDash \bigwedge \Lambda \to \varphi$

Let $\Lambda \vDash \varphi$

$$
\begin{aligned}
&\Lambda \vDash \varphi \\
&\iff \text{for any } v, \text{ if } v \vDash \bigwedge \Lambda \text{ then } v \vDash \varphi
\end{aligned}
$$

$$
\begin{aligned}
&\text{Case I: Let } v \text{ be an assignment such that } v \vDash \bigwedge \Lambda \\
&\implies v \vDash \varphi \\
&\implies v \vDash \bigwedge \Lambda \text{ and } v \vDash \varphi \\
&\implies v \vDash \bigwedge \Lambda \to \varphi
\end{aligned}
$$

$$
\begin{aligned}
&\text{Case II: Let } v \text{ be an assignment such that } v \nvDash \bigwedge \Lambda \\
&\implies v \vDash \bigwedge \Lambda \to \varphi \text{ vacously}
\end{aligned}
$$

Therefore by cases I and II (or by any assignment of Λ),

$$\Lambda \vDash \varphi \implies \vDash \bigwedge \Lambda \to \varphi$$

RTP: $\vDash \bigwedge \Lambda \to \varphi \implies \Lambda \vDash \varphi$

```
homework :)
```
